Recalling what I said about the explicit dependencies principle, you may be able to discern why the service locator is an anti-pattern. It is not clear from the code in listing 7.17 what the dependencies of the `IndexModel` are—apart from the service provider. In fact, it still depends on the `ICityService`, but that detail is no longer visible to code outside of the class.

The service provider is also available as a request feature (http://mng.bz/neE2), so you don't even need to inject the provider into classes that have access to the `HttpContext`. You could replace the line of code that resolves the city service in listing 7.17 with the following:

```
var cityService =
➥ HttpContext.RequestServices.GetRequiredService<ICityService>();
```

Dependency injection and the other terms that accompany it sound complicated, but the reality is that it is a pretty simple technique that helps to achieve high-quality code. The built-in service container should suffice for most use cases, but if you find yourself needing something more advanced, you can use one of the many (usually free and open source) third-party containers that support ASP.NET Core. Integration is usually quite straightforward and should be fully documented by the vendor.

In the next chapter, we will look at working with data in a Razor Pages application. While we do that, we will create a new service that obtains data from a database and seamlessly swap out the existing service for a new one, demonstrating one of the key advantages of using DI.

Summary

- Dependency injection (DI) is a key feature in ASP.NET Core.